

# 이벤트 시스템 — 안 4 구현 롤아웃 (단계별 공유·검증)

선행 안: 이벤트 시스템 구현 - 안 4 (kind 분리 + chain).md

목적: 안 4 의 §5 단계 분할과 §6 검증 시나리오를 실제 작업 → 검증 → 공유 → 다음 단계 진입 흐름으로 매핑. 각 단계의 작업 범위·검증 명령·공유 결과물을 사전 합의 후 코드 진입.

## §0. 단계 개요

| 단계         | 산출물                        | 검증 방식         | 공유 형태             |
|------------|----------------------------|---------------|-------------------|
| 1A         | EventManager + dialogue 시물 | LimboConsole  | 콘솔 세션 로그 + diff   |
| 1B         | event_ui.gd + main.tscn    | 실 플레이         | 짧은 영상/스크린샷 + diff |
| 2A (다음 차로) | effect kind                | 콘솔 (UI 입력 없음) | 콘솔 로그             |
| 2B (다음 차로) | chain ( next )             | 콘솔 + 실 플레이    | 콘솔 로그             |
| 2C (다음 차로) | movie kind                 | 실 플레이 (자산 의존) | 영상                |
| 2D (다음 차로) | choice kind                | 콘솔 + 실 플레이    | 콘솔 + 영상           |

MVP = 1A + 1B. 그 이후는 차로별 독립 진입.

## §1. 흐름 규칙

- 각 단계 진입 전: 본 문서의 해당 §X 작업·검증 부분 사용자와 합의. 변경 시 명세 갱신 후 진입.
- 각 단계 종료 시: 자가점검 체크리스트 → 공유 결과물 (콘솔 로그 / diff / 영상) → 사용자 OK 후 다음 단계.
- 단계 간 별도 브랜치·PR 안 만들. 안 4 §8 의 커밋 단위 그대로.
- 막히면 즉시 보고. 명세 변경 동반 시 안 4 갱신 → 본 문서 갱신 → 재진입.

## §2. 단계 1A — EventManager + dialogue 시물

### 작업 범위

신규 파일 - event\_manager.gd (autoload) - event\_state: Dictionary - 시그널:  
event\_state\_changed , event\_resolved(result: Dictionary) - begin\_event(event\_id: String, context: Dictionary) -> void - advance\_line() -> void - \_finalize\_event() ->

void - `_resolve_event_for_node(node: Dictionary) -> String` (트리거 + once\_per + 가중치) -  
LimboConsole 등록: `event_advance` / `show_event` / `event_force_begin`

수정 - `project.godot` — EventManager 오토로드 등록 - `game_data.gd` - `EVENTS` 상수 (dialogue kind 1~2개) - `TEST_MAP_GRAPH` 의 노드 1개에 `"type": "event"` 부여 - `run_manager.gd` -  
`_new_big_run` 에 `"seen_events": {}` 초기화 - `move_to_node` 의 type 분기에 `"event"` — phase 전  
이 + `EventManager.begin_event` 호출 - `_ready` 에  
`EventManager.event_resolved.connect(_on_event_resolved)` - `_on_event_resolved` 핸들러 —  
`seen_events` 갱신 + phase 복귀

## 검증 시나리오 — LimboConsole 만

가정: 4 번 노드를 type `"event"` 로 변경, `EVENTS` 에 `intro_monologue` (dialogue, lines 2 개, once\_per: big\_run) 등록.

```
> show_run
{phase: "map", current_node_id: 1, ...}

> show_big_run
{seen_events: {}, ...}

> # 이벤트 노드로 이동
> # (UI 가 아직 없으므로 콘솔로 직접 트리거. 1B 단계에서 자연스러워짐)
> move 4

> show_run
{phase: "event", current_node_id: 4, ...}

> show_event
{event_id: "intro_monologue", kind: "dialogue",
 phase: "awaiting_input", chain_root_id: "intro_monologue", line_idx: 0}

> # 이벤트 중 외부 이동 시도 → 거부
> move 5
[warn] move_to_node: phase=="event" - 이벤트 중 이동 불가

> event_advance
> show_event
{... line_idx: 1}

> # 마지막 라인 advance → 종료
> event_advance
[event_resolved] {event_id: "intro_monologue"}

> show_run
{phase: "map", ...}

> show_big_run
```

```
{seen_events: {intro_monologue: 1}, ...}

> # once_per "big_run" 필터 검증 - 같은 큰 런 내 재진입 시 발생 안 함
> move 1 (인접 복귀)
> move 4
> show_run
{phase: "map", ...} # 이벤트 미발생, 일반 노드처럼 통과

> # 회귀 후 검증
> end_run cleared
> show_big_run
{seen_events: {intro_monologue: 1}, ...} # 회귀 통과해 유지

> move 4
> show_run
{phase: "map", ...} # 여전히 안 발생 (once_per: big_run 이므로 회귀 무관)
```

## 자가점검 체크리스트

- [] `event_state` 비활성 시 {} 보장
- [] `phase map → event → map` 전이 정확
- [] `seen_events` 카운트 정확
- [] `once_per: "big_run"` 필터 작동
- [] 이벤트 활성 중 외부 `move_to_node` 거부 (`push_warning + return false`)
- [] `event_ui.gd` 없는 상태에서도 시물 일관 작동
- [] 모르는 kind 만나면 `push_warning` + 즉시 `_finalize_event` (스텝)

## 공유 결과물

- LimboConsole 세션 로그 (위 시퀀스 결과)
- 변경된 파일 목록 + 핵심 함수 diff 요약

## §3. 단계 1B — event\_ui.gd + main.tscn 배선

### 작업 범위

신규 파일 - `event_ui.gd` (Control 스크립트) - `_ready` :

`EventManager.event_state_changed.connect(_on_event_state_changed)` -

`_on_event_state_changed` :- `event_state.is_empty()` → 화면 숨김 - `kind == "dialogue"` → 라인 렌더 (`EVENTS[event_id].lines[line_idx]`) - 그 외 kind → 빈 화면 (다음 차로에서 분기 추가) - 진행 버튼 또는 화면 클릭 → `EventManager.advance_line()`

수정 - `main.tscn` — `run_ui` 자식으로 `event_screen: Control` 추가, `event_ui.gd` 부착 - `run_ui.gd::screens` 에 `"event": $event_screen` 추가

## 검증 시나리오 — 실 플레이

1. 새 게임 시작 → 맵 화면.
2. event 노드 (4번) 클릭 → 이벤트 화면 자동 노출, 라인 1 표시 (speaker · text).
3. 화면 클릭 (또는 진행 버튼) → 라인 2 표시.
4. 마지막 라인 클릭 → 화면 사라지고 맵 복귀.
5. 이벤트 진행 중 맵 영역 클릭 시도 → 차단됨 (event\_screen 이 위 또는 run\_ui.show\_phase 가 맵 숨김).
6. 같은 큰 런에서 다시 4번 노드 클릭 → 이벤트 미발생, 일반 노드처럼 통과.

## 자가점검 체크리스트

- [ ] 실 플레이 흐름 1~6 모두 정상
- [ ] `event_ui.gd` 삭제 후 시뮬 컴파일 에러 0건 (안 4 §1 검증의 마지막 항목)
- [ ] 이벤트 중 맵 클릭 시 RunManager 입력 무시
- [ ] `_displayed_lines` 같은 UI 자체 누적 상태가 회차 사이 안 넘어감 (event\_state 가 사라지면 UI 도 초기화)

## 공유 결과물

- 짧은 플레이 영상 (10~20 초) 또는 단계별 스크린샷 4~5 장
- `event_ui.gd` / `run_ui.gd` / `main.tscn` diff

---

## §4. 단계 2A — effect kind 디스패처 (다음 차로)

### 작업 범위

- `event_manager.gd::begin_event` 의 kind 분기에 `"effect"` 추가.
- `_dispatch_effect(def: Dictionary)` :
- `event_state = {... kind: "effect", phase: "running"}`
- `effects` 배열 순회 → `RunManager._apply_<type>(params)` 호출
- 모르는 type → `push_warning` , 건너뛰
- 순회 끝 → 즉시 `_finalize_event()`
- `game_data.gd::EVENTS` 에 effect kind 이벤트 1개 (예: 데이터 회수 +10).

### 검증 — 콘솔

```

> show_big_run
{research_data: 0, seen_events: {}, ...}

> move 4    (effect kind 노드)

> show_event
{}    # 이미 _finalize 까지 동기 완료

> show_run
{phase: "map", ...}    # 즉시 복귀

> show_big_run
{research_data: 10, seen_events: {data_pickup: 1}, ...}

```

## 자가점검

- [] `_apply_<type>` 단일 경로 유지 (escape 0)
- [] UI 입력 없이 한 프레임 내 종료
- [] effect kind 도 `once_per` 정상
- [] 모르는 effect type → 경고 + 건너뛴, 이벤트는 정상 종료

## 공유

- 콘솔 로그 + before/after 비교

## §5. 단계 2B — chain ( `next` 필드)

### 작업 범위

- EventDefinition 에 optional `next: String` 필드 추가.
- `event_manager.gd::_finalize_event` 직전 분기:
- 현재 정의의 `next` 존재 + `EVENTS[next]` 존재 → `event_state.event_id = next` 갱신, kind 별 재 초기화 (line\_idx = 0 등), `event_state_changed.emit()` . **event\_resolved emit 안 함, phase 안 빠짐.**
- 없거나 chain 끝 → 기존 `_finalize_event` 그대로.
- `chain_root_id` 는 chain 의 첫 `begin_event` 에서 고정. 이후 노드 전이 시 그대로 유지.
- `seen_events` 카운트는 chain root 만 ( `_on_event_resolved` 가 `result.event_id = chain_root_id` ).

### 검증 — 콘솔

가정: A (dialogue, 라인 2) → next: B → B (effect, body\_hp -5) → next: C → C (dialogue, 라인 1) → next 없음.

```

> show_run
{phase: "map", ...}, body_hp: 150

> move 4

> show_event
{event_id: "A", kind: "dialogue", chain_root_id: "A", line_idx: 0}

> event_advance (A 라인 1)
> event_advance (A 라인 2 - A 끝)
> show_event
{event_id: "B", kind: "effect", chain_root_id: "A"} # B 로 자동 전이, root 는 A
> show_run
{phase: "event"} # phase 안 빠짐

> # B 는 effect 라 한 프레임 내 자동 통과 → C 로 전이
> show_event
{event_id: "C", kind: "dialogue", chain_root_id: "A", line_idx: 0}
> show_run
{phase: "event", body_hp: 145} # B 의 효과 적용됨

> event_advance (C 라인 1 - chain 끝)
[event_resolved] {event_id: "A"} # root 만

> show_run
{phase: "map", body_hp: 145}

> show_big_run
{seen_events: {A: 1}} # A 만 카운트, B/C 없음

```

## 자가점검

- [] chain 진행 동안 `phase == "event"` 유지
- [] `chain_root_id` 가 chain 진행 내내 고정
- [] `seen_events` 는 root 한 개만 카운트
- [] chain 도중 외부 `move_to_node` 거부 유지
- [] 잘못된 `next` (EVENTS 미등록) → `push_warning` + 정상 종료

## 공유

- 콘솔 로그

## §6. 단계 2C — movie kind

자산 의존 단계. 짧은 더미 영상 ( `.ogv` ) 준비 후 진입.

## 작업 범위

- `event_manager.gd::begin_event` 분기에 `"movie"` 추가.
- `_dispatch_movie(def) :`
- `event_state = {... kind: "movie", phase: "playing"}`
- `event_state_changed.emit()` → UI 가 `VideoStreamPlayer` 띄움
- `event_ui.gd` 에 movie 분기 — `VideoStreamPlayer` 인스턴스화, `movie.path` 로드, 재생.
- 재생 종료 시그널 (`finished`) → `EventManager._finalize_event` 호출.
- `skippable: true` 면 화면 클릭 → 즉시 종료.

## 검증 — 실 플레이

- movie 이벤트 노드 진입 → 영상 재생 → 끝/스킵 → chain next 또는 종료.

## 공유

- 짧은 플레이 영상
- 

## \$7. 단계 2D — choice kind

---

### 작업 범위

- `event_manager.gd::begin_event` 분기에 `"choice"` 추가.
- `_dispatch_choice(def) :`
- `event_state = {... kind: "choice", phase: "awaiting_input", prompt, choices}`
- `event_state_changed.emit()`
- 신규 메서드 `select_choice(idx: int) -> void :`
- 유효성 검사 → `event_state.event_id = choices[idx].next` , kind 재초기화 → `event_state_changed.emit()` (chain 전이)
- `event_ui.gd` 에 choice 분기 — prompt 표시 + 버튼 N개. 클릭 → `select_choice(idx)` .
- LimboConsole: `event_choose <idx>` .

### 검증

- prompt 표시 → 두 가지 → 가지 A 선택 시 그 `next` 로 / B 선택 시 다른 `next` 로.
- chain root 는 choice 이벤트의 id 로 고정.

### 공유

- 실 플레이 영상 + 콘솔 로그
-

## §8. 통합 산출물 (최종 — MVP + 다음 차로)

---

MVP 종료 시점 (1A + 1B): - 신규: `event_manager.gd`, `event_ui.gd` - 변경: `game_data.gd`, `run_manager.gd`, `run_ui.gd`, `main.tscn`, `project.godot`

다음 차로 누적 (2A~2D): - 변경: 위 파일들의 dispatcher / 분기 / EVENTS 풀 확장 - 자산:

`res://cutscenes/*.ogv` (movie kind 시점)

---

## §9. 자가점검 — 단계 진입 전 공통 점검

---

각 단계 진입 직전 항상 확인:

- [ ] 본 문서의 해당 §X 작업·검증 부분 변경된 부분 없는지 사용자 합의
  - [ ] 이전 단계 자가점검 모두 통과 상태인지
  - [ ] 안 4 의 §0 확정 사항 (A~H) 와 충돌 없는지
  - [ ] 메모리 — RunManager 단일 추적점 정책 / 새 매니저 분리 기준 (lifecycle + discrete structure) 깨지지 않는지
- 

§0 의 단계 표 + §1 흐름 규칙 합의 후 1A 진입.